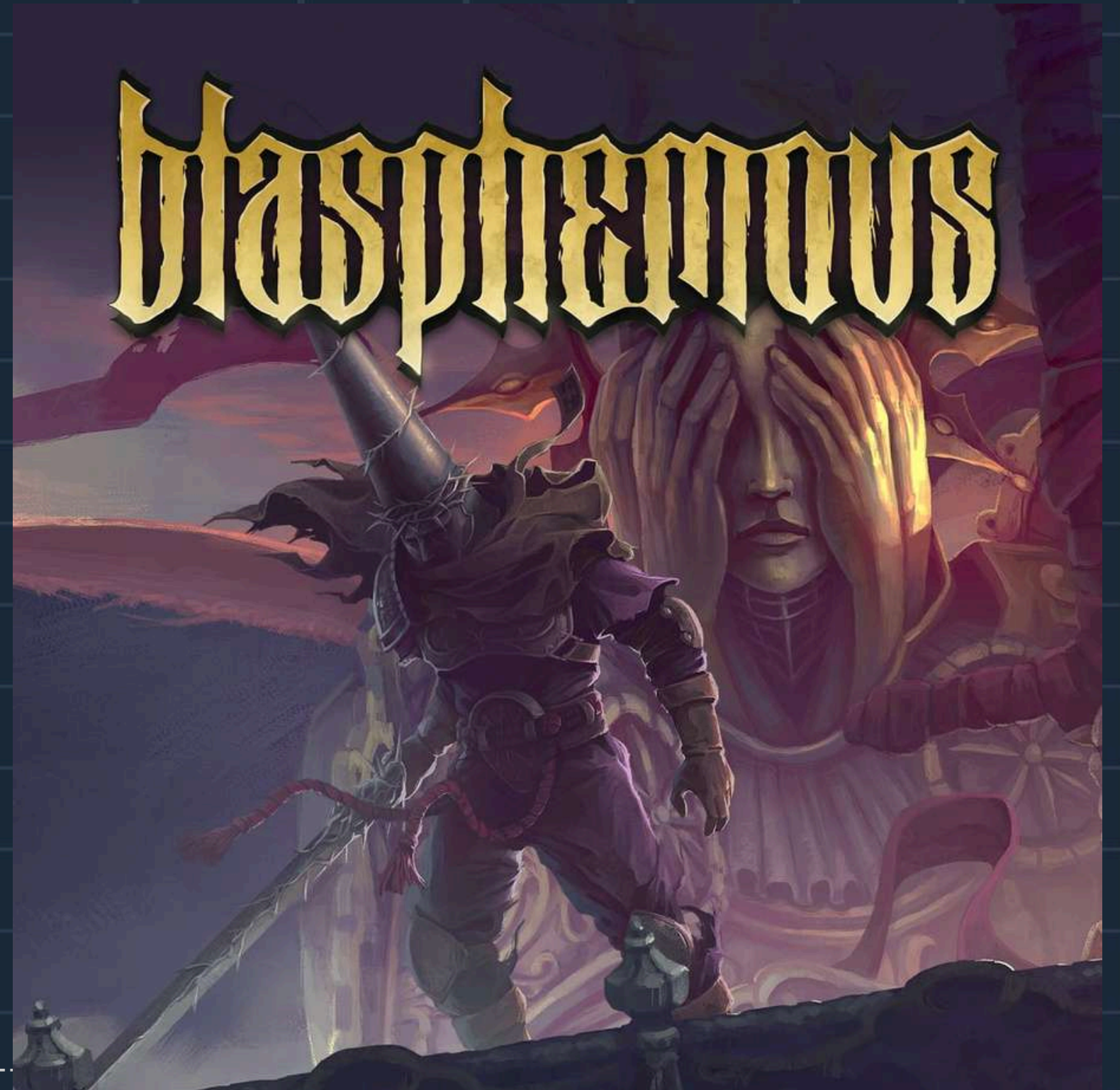


```
// Welcome to the Blasphemous Clone Project  
console.log("Developer's Programming Report");
```

{블래스퍼머스; 모작 프로젝트

```
// Ready to Start?  
console.log("Next Page");
```



목차

- index
 - 00_Index Page
 - 01_ Project Overview
 - 02_Game Introduction
 - 03_Technology Stack
 - 04_Implemented Features
 - 05_Detailed Design

```
// Table of Contents
const contents = [
```

contents 01

프로젝트 개요

"Project Overview"

contents 02

게임 소개

"Introduction"

contents 03

기술 스택

"Technology Stack"

contents 04

구현 기능

"Primary Features"

contents 05

상세 설계

"Detailed Design"

프로젝트 개요

index

00_Index Page

01_Project Overview

02_Game Introduction

03_Technology Stack

04_Implemented Features

05_Detailed Design

<!-- -->

<!-- "프로젝트 개요" -->

<!-- 프로젝트명 -->

◦ 블래스퍼머스 모작 프로젝트

<!-- 개발일정 -->

◦ 2025.11.25 ~ 2025.12.15

<!-- 개발특징 -->

◦ **WinApi-GameFramework** 사용

◦ 원작 리소스 활용

◦ 콤보 공격, 패리, 회피 슬라이드 등의 핵심 전투 시스템 구현

게임 소개

index

00_Index Page

01_Project Overview

02_Game Introduction

03_Technology Stack

04_Implemented Features

05_Detailed Design

// Game Features



Features

- 어두운 세계관 **메트로배니아** 게임
- 점프, 사다리타기, 매달리기 활용 탐험
- 콤보, 패링, 회피(슬라이딩) 활용 전투

// 자료 출처 : 블래스퍼머스 스팀 커뮤니티

기술 스택

Programming Plan

00_Index Page

01_Project Overview

02_Game Introduction

03_Technology Stack

04_Implemented Features

05_Detailed Design

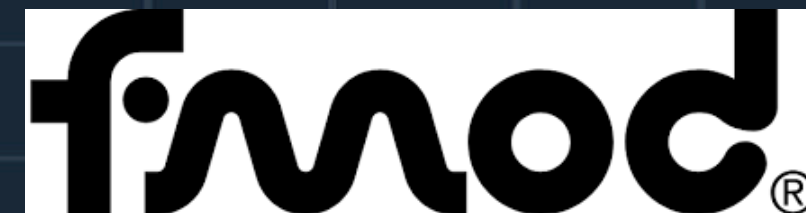
```
<!---->  
<!-- "기술스택 안내" -->
```

jungtaek6681/WinApi-
GameFramework

WinApi GameFramework



```
// https://github.com/jungtaek6681/WinApi-GameFramework/tree/master
```



구현 기능

index

00_Index Page

01_Project Overview

02_Game Introduction

03_Technology Stack

04_Implemented Features

05_Detailed Design

<!---->

<!-- "핵심 구현 기능" -->

<전투 시스템>

애니메이션 이벤트 델리게이트로 타이밍 판정

<이벤트> 애니메이션 이벤트 정보를 구독자에 발행 → 판정 타이밍을 애니메이션에 동기화
<콤보> 판정 구간 내 입력 → 선입력 저장 → 다음 콤보 연결
<패리> 판정 구간 내 피격 → 패리 성공 → 카운터 연계

<지형 시스템>

콜라이더 감지, 위치 스냅/보정

<지면 착지> 충돌 시 겹친 크기 계산 → 수직/수평 중 작은 쪽으로 밀어내어 보정
<낙하 터널링 보정> 이전~현재 프레임 발 위치 사이를 Trace → 통과한 지면에 스냅 (통과를 허용한 플랫폼이 아닌 경우)

<AI 시스템>

거리 기반 행동 결정 (추격 / 공격 / 대기)

<상태 전환> 감지 범위 진입 시 순찰 → 추격, 공격 범위 진입 시 공격 실행
<보스 패턴> 가중치 + 랜덤 선택 기반 공격 패턴 선택
<최적화> AI 업데이트 주기 조절, 화면 밖 업데이트 스킵

상세 설계 (1)

index

00_Index Page

01_Project Overview

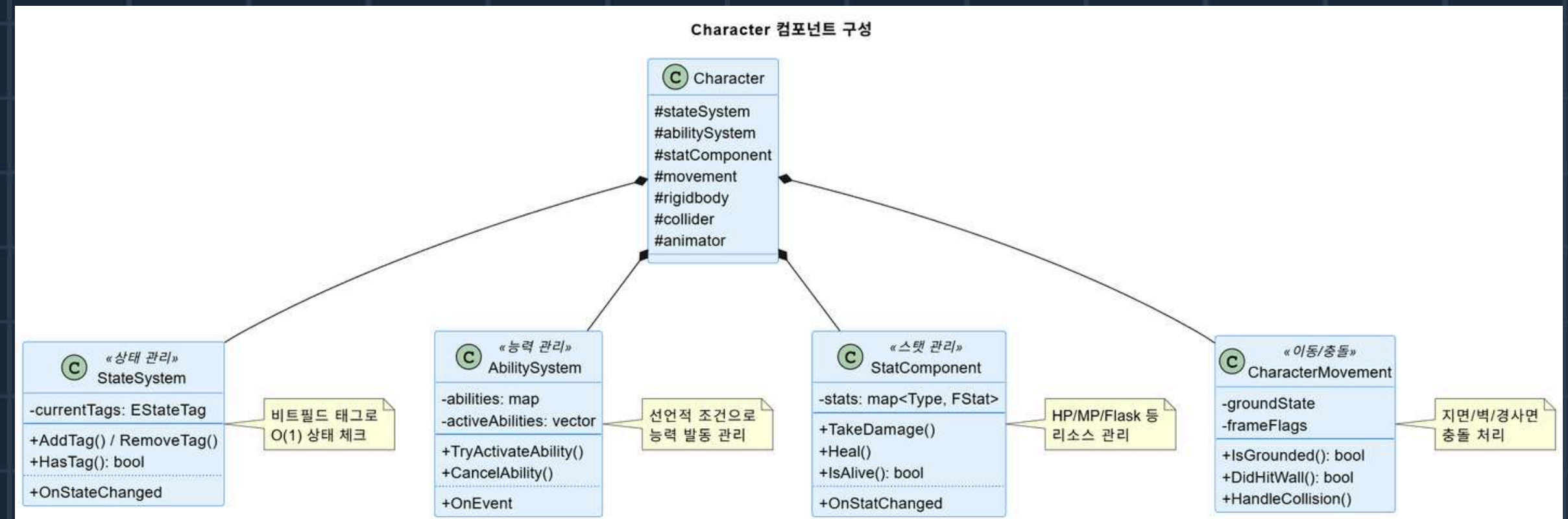
02_Game Introduction

03_Technology Stack

04_Implemented Features

05_Detailed Design

// Components



<!-- "상속보다 조합" -->

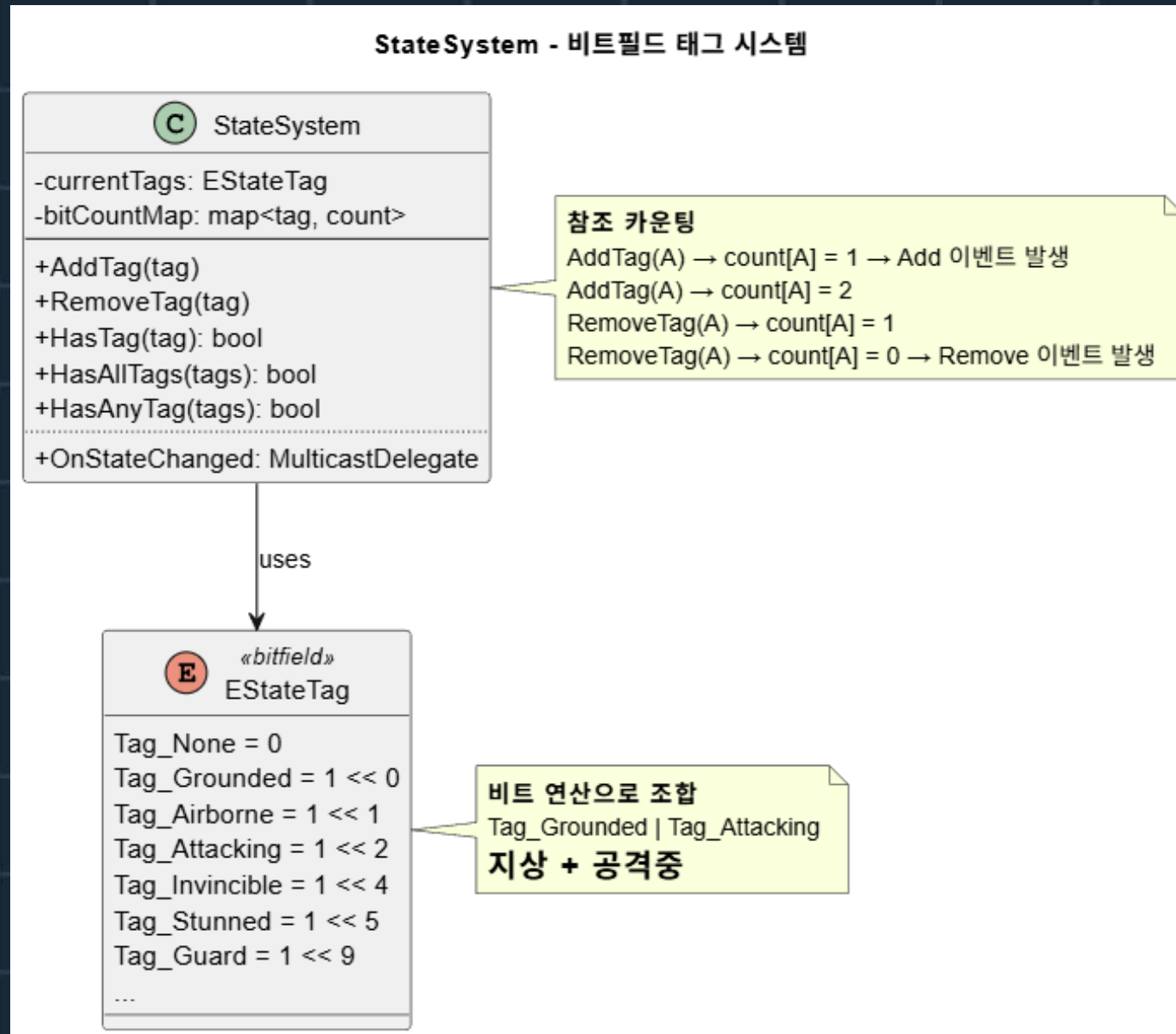
새로운 캐릭터 추가 시 기존 컴포넌트 재사용, Ability만 새로 정의.

상세 설계 (2)

index

- 00_Index Page
- 01_Project Overview
- 02_Game Introduction
- 03_Technology Stack
- 04_Implemented Features
- 05_Detailed Design

// StateSystem



<!-- "한계" -->

비트 플래그 기반이므로 상태 태그 개수 제한

<!-- "비트필드 태그로 상태 관리" -->

비트 OR로 복합 상태 표현:

`Tag\_Grounded | Tag\_Attacking`

→ 조건문과 bool 플래그 난립 방지

<!-- "참조 카운팅" -->

같은 태그를 여러 곳에서 추가할 수 있음

(예: 여러 Ability가 `Tag\_BlockMovement` 추가)

→ 카운트가 0이 되어야 실제 태그 제거

<!-- "상태 변경 이벤트" -->

`OnStateChanged` 델리게이트로 상태 변경
시 구독자에게 알림

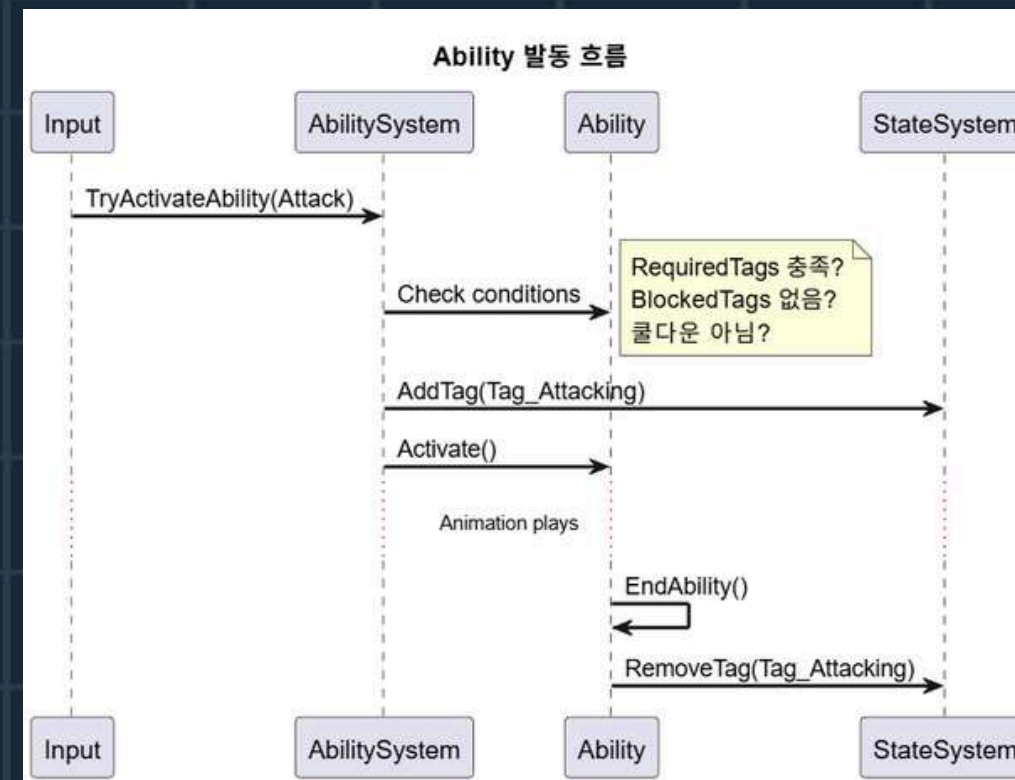
→ oldTags, newTags 비교로 상태 전환에 따
른 특수 로직 정의 가능

상세 설계 (3)

index

- 00_Index Page
- 01_Project Overview
- 02_Game Introduction
- 03_Technology Stack
- 04_Implemented Features
- 05_Detailed Design

// AbilitySystem



<!-- "선언적 발동 조건 정의" -->

- > 상태 태그 기반 어빌리티 발동 조건 검사
- > 발동 중 태그 추가, 종료 후 태그 제거
- 새 Ability 정의 시 if문 없이 태그만 설정
- 시스템이 알아서 처리

Example

```
class Ability_ComboAttack : public Ability
{
public:
    Ability_ComboAttack();

    /*~ Ability Interface ~*/
    EStateTag GetRequiredTags() const override { return Tag_Grounded; }
    EStateTag GetBlockedTags() const override { return Tag_BlockAbility | Tag_Hit; }
    EStateTag GetTagsToAdd() const override { return Tag_Attacking | Tag_BlockMovement | Tag_AbilityAnimation; }
```

상세 설계 (4)

index

00_Index Page

01_Project Overview

02_Game Introduction

03_Technology Stack

04_Implemented Features

05_Detailed Design

// Event 기반 설계

Example : Ability_ComboAttack

```
// 이벤트 바인딩
WaitEvent(EGameEvent::Input_Attack_Pressed, callback: ⚡ BIND_EVENT(this, OnInputAttack));
WaitEvent(EGameEvent::HitCheck, callback: ⚡ BIND_EVENT(this, OnHitCheck));
WaitEvent(EGameEvent::ComboWindowOpen, callback: ⚡ BIND_EVENT(this, OnComboWindowOpen));
WaitEvent(EGameEvent::ComboWindowClose, callback: ⚡ BIND_EVENT(this, OnComboWindowClose));

// 공격 1타 시작
Attack();
```

<!-- "Delegate 패턴" -->

> Ability가 특정 이벤트를 구독하고, 이벤트 발생 시 콜백 실행

> `SafeDelegateHandle`: RAII 방식 자동 해제로 null 크래시 및 메모리 누수 방지

<!-- "전역 이벤트 버스" -->

> `OnPlaySFX`, `OnSpawnVFX`, `OnCameraShake` 등 전역 이벤트

> Ability가 직접 Manager를 참조하지 않고 이벤트만 발행 → 느슨한 결합

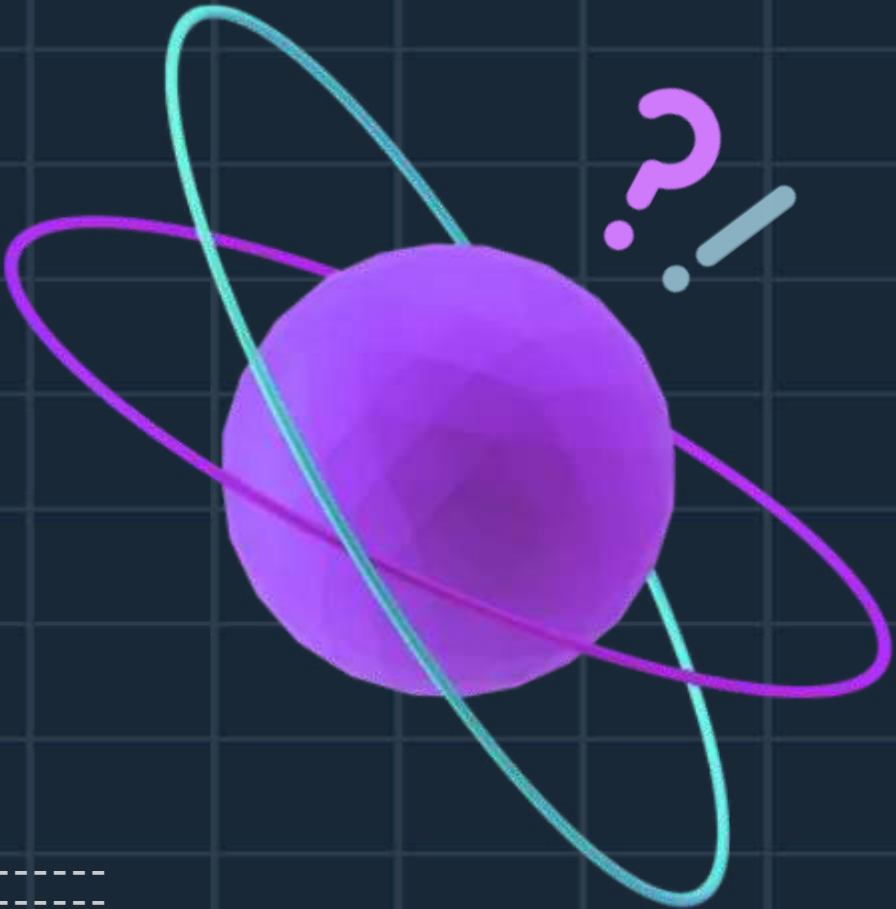
End

index

- 00_Index Page
- 01_Project Overview
- 02_Game Introduction
- 03_Technology Stack
- 04_Implemented Features
- 05_Detailed Design

<!---->

감사합니다!



{thank you;